

MOMO SMS DATABASE DESIGN

Team members

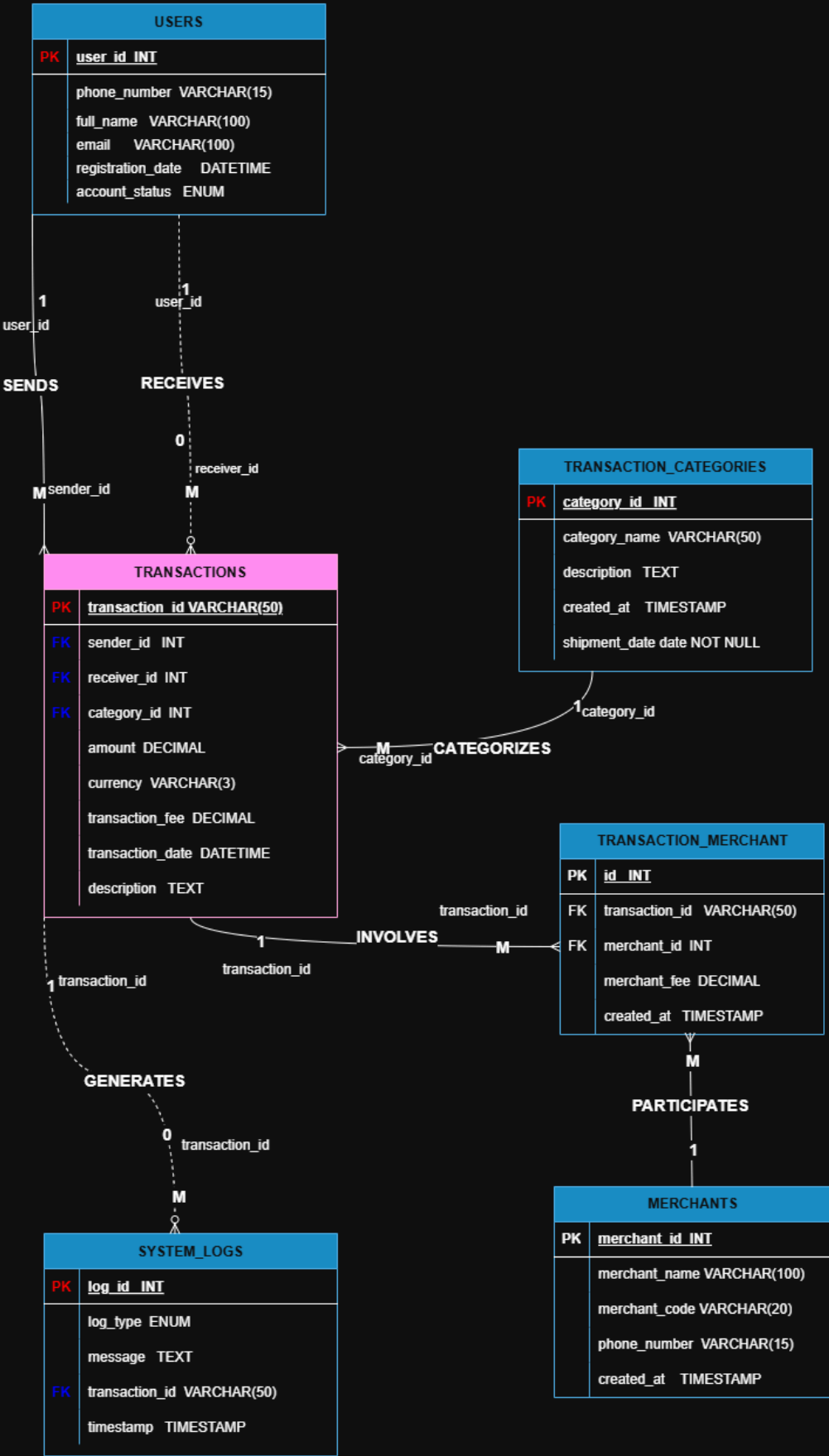
1. Dianah Shimwa Gasasira
2. Ayobamidele Aiyedogbon
3. Jesse Nkubito
4. Bior Aguer Kuir

Overview

Our MoMo SMS data processing system has a six-entity relational database design that balances normalization, data integrity, and business requirements.

ERD DIAGRAM

MOMO SMS DATABASE ERD



Entity uses and functions

Users Table: Stores customer account information for all MoMo users.

Transaction_Categories: Defines and categorizes different types of mobile money transactions.

Merchants: Stores information about businesses and merchants accepting MoMo payments.

Transactions: Main table that records all mobile money transaction details.

Transaction_Merchant Junction Table: Resolves the many-to-many relationship between Transactions and Merchants.

System_Logs: Tracks ETL processing events and system activities for monitoring and debugging.

DATA DICTIONARY

TABLE 1 : USERS

COLUMN NAME	DATA TYPE	KEY	NULLABLE	DEFAULT	DESCRIPTION
user_id	int	PK	no	auto_increment	Unique identifier for each user
phone_number	varchar(15)	UNIQUE	no	-	User's phone in international format e.g +27078000000
full_name	varchar(100)	-	no	-	User's complete name
email	varchar(100)	UNIQUE	yes	null	User's email address (optional)
registration_date	datetime	-	no	current_timestamp	Date and time when account was created
account_status	enum	-	no	active	Account status: active, suspended, or closed

Constraints:

- Primary Key: user_id
- Unique: phone_number, email
- Index: phone_number, account_status, email

TABLE 2: transaction_categories

Column Name	Data Type	Key	Nullable	Default	Description
category_id	INT	PK	No	auto_increment	Unique identifier for each category
category_name	varchar(50)	UNIQUE	No	-	Category display name eg money transfer, loan
description	TEXT	-	Yes	NULL	Detailed explanation of the category
created_at	TIMESTAMP	-	No	current_timestamp	Record creation timestamp

Constraints:

- Primary Key: category_id
- Unique: category_name

TABLE 3: Merchants

Column Name	Data Type	Key	Nullable	Default	Description
merchant_id	INT	PK	No	AUTO_INCREMENT	Unique identifier for each merchant
merchant_name	varchar(100)	-	No	-	Merchant name

merchant_code	varchar(20)	UNIQUE	No	-	Unique code assigned to merchant
phone_number	varchar(15)	-	No	-	Merchant contact number
created_at	TIMESTAMP	-	No	CURRENT_TIMESTAMP	Record creation timestamp

Constraints:

- Primary Key: merchant_id
- Unique: merchant_code
- Index: merchant_code, phone_number

TABLE 4: Transactions

Column Name	Data Type	Key	Nullable	Default	Description
transaction_id	varchar(50)	PK	No	-	Unique identifier for transactions
sender_id	INT	FK	No	-	User initiating the transaction
receiver_id	INT	FK	Yes	NULL	User receiving money
category_id	INT	FK	No	-	Type of transaction
amount	DECIMAL(15,2)	-	No	-	Transaction amount in currency units
currency	varchar(3)	-	No	'RWF'	Currency code
transaction_fee	DECIMAL(10,2)	-	No	0.00	Fee charged for the transaction
transaction_date	DATETIME	-	No	-	When the transaction occurred
description	TEXT	-	Yes	NULL	Additional notes

Constraints:

- Primary Key: transaction_id
- Foreign Keys:
 - sender_id → Users(user_id) [ON DELETE RESTRICT, ON UPDATE CASCADE]
 - receiver_id → Users(user_id) [ON DELETE RESTRICT, ON UPDATE CASCADE]
 - category_id → Transaction_Categories(category_id) [ON DELETE RESTRICT, ON UPDATE CASCADE]
- Check Constraints:
 - amount > 0
 - transaction_fee >= 0
 - sender_id ≠ receiver_id (when receiver_id is not NULL)
- Index: sender_id, receiver_id, category_id, transaction_date, amount

TABLE 5: Transaction_merchant

Column Name	Data Type	Key	Nullable	Default	Description
id	INT	PK	No	AUTO_INCREMENT	Unique identifier for this relationship record
transaction_id	varchar(50)	FK	No	-	Reference to transaction
merchant_id	INT	FK	No	-	Reference to merchant
merchant_fee	DECIMAL(10,2)	-	No	0.00	Fee to be paid to merchant
created_at	TIMESTAMP	-	No	CURRENT_TIMESTAMP	When relationship was created

Constraints:

- Primary Key: id
- Foreign Keys:
 - transaction_id → Transactions(transaction_id) [ON DELETE CASCADE, ON UPDATE CASCADE]

- merchant_id → Merchants(merchant_id) [ON DELETE CASCADE, ON UPDATE CASCADE]
- Unique: (transaction_id, merchant_id) combination
- Check Constraint: merchant_fee >= 0
- Index: transaction_id, merchant_id

Purpose of M:N Relationship:

- One transaction can involve multiple merchants (split payments)
- One merchant can participate in multiple transactions
- Stores merchant-specific fees for each transaction

TABLE 6: System_Logs

Column Name	Data Type	Key	Nullable	Default	Description
log_id	INT	PK	No	AUTO_INCREMENT	Unique identifier for each log entry
log_type	enum	-	No	-	Severity level ie warning, info, error
message	TEXT	-	No	-	Log message content
transaction_id	varchar(50)	FK	Yes	NULL	Related transaction if applicable
timestamp	TIMESTAMP	-	No	CURRENT_TIMESTAMP	When log entry was created
additional_data	JSON	-	Yes	NULL	Extra context data in JSON format

Constraints:

- Primary Key: log_id
- Foreign Key: transaction_id → Transactions(transaction_id) [ON DELETE SET NULL, ON UPDATE CASCADE]
- Index: log_type, timestamp, process_name, transaction_id

Next up is the queries and the screenshots of the outputs

QUERY 1: Viewing all users

```
mysql> SELECT * FROM users;
```

user_id	phone_number	full_name	email	registration_date	account_status
1	250788000001	Alice Kamanzi	alice.kamanzi@example.com	2025-01-10 08:15:00	active
2	250788000002	Brian Mugisha	brian.mugisha@example.com	2025-01-11 09:30:00	active
3	250788000003	Claire Nsengiyumva	claire.nsengiyumva@example.com	2025-01-12 10:45:00	active
4	250788000004	Daniel Habimana	daniel.habimana@example.com	2025-01-13 11:20:00	suspended
5	250788000005	Eve Uwamahoro	eve.uwamahoro@example.com	2025-01-14 12:05:00	active

5 rows in set (0.10 sec)

QUERY 2: Viewing all transactions data

```
mysql> SELECT transaction_id, sender_id, receiver_id, category_id, amount, currency, transaction_fee, transaction_date, description FROM transactions ORDER BY transaction_date DESC;
```

transaction_id	sender_id	receiver_id	category_id	amount	currency	transaction_fee	transaction_date	description
txn-0005	5	NULL	5	2000.00	RWF	0.00	2025-01-18 08:45:00	Transfer to savings at SafeBank
txn-0004	4	NULL	4	15000.00	RWF	100.00	2025-01-17 18:20:00	Movie tickets at CinemaPlus
txn-0003	3	NULL	3	3000.00	RWF	20.00	2025-01-16 14:05:00	Taxi payment via FastTaxis
txn-0002	2	NULL	2	125000.00	RWF	500.00	2025-01-16 10:30:00	Electricity bill payment to City Electricity
txn-0001	1	NULL	1	45250.00	RWF	250.00	2025-01-15 09:00:00	Grocery purchase at Kigali Supermarket

5 rows in set (0.04 sec)

QUERY 3: Viewing transactions of customer names who sent and received money

```
mysql> SELECT t.transaction_id, sender.full_name AS sender, receiver.full_name AS receiver, t.amount FROM transactions t JOIN users sender ON t.sender_id = sender.user_id LEFT JOIN users receiver ON t.receiver_id = receiver.user_id;
```

transaction_id	sender	receiver	amount
txn-0001	Alice Kamanzi	NULL	45250.00
txn-0002	Brian Mugisha	NULL	125000.00
txn-0003	Claire Nsengiyumva	NULL	3000.00
txn-0004	Daniel Habimana	NULL	15000.00
txn-0005	Eve Uwamahoro	NULL	2000.00

5 rows in set (0.01 sec)

QUERY 4: Viewing which merchants are involved in which transactions (M:N relationship)

```
mysql> SELECT t.transaction_id, t.amount, m.merchant_name FROM transactions t JOIN transaction_merchant tm ON t.transaction_id = tm.transaction_id JOIN merchants m ON tm.merchant_id = m.merchant_id ORDER BY t.transaction_id;
```

transaction_id	amount	merchant_name
txn-0001	45250.00	Kigali Supermarket
txn-0002	125000.00	City Electricity
txn-0003	3000.00	FastTaxis
txn-0004	15000.00	CinemaPlus
txn-0005	2000.00	SafeBank

5 rows in set (0.09 sec)

QUERY 5: Count transactions by category

```
mysql> SELECT tc.category_name, COUNT(*) AS total_transactions, SUM(t.amount) AS total_amount FROM transactions t JOIN transaction_categories tc ON t.category_id = tc.category_id GROUP BY tc.category_name ORDER BY total_amount DESC;
+-----+-----+-----+
| category_name | total_transactions | total_amount |
+-----+-----+-----+
| Utilities     | 1                 | 125000.00    |
| Groceries     | 1                 | 45250.00     |
| Savings       | 1                 | 20000.00     |
| Entertainment | 1                 | 15000.00     |
| Transport     | 1                 | 3000.00      |
+-----+-----+-----+
5 rows in set (0.06 sec)
```

QUERY 6: Viewing total transactions

```
mysql> SELECT SUM(amount) AS total_money FROM transactions;
+-----+
| total_money |
+-----+
| 208250.00  |
+-----+
1 row in set (0.02 sec)
```

RELATIONSHIP TABLES

Table	Column	References	Relationship Type	Description
transactions	sender_id	users.user_id	many-to-one	Each transaction is sent by one user
transactions	receiver_id	users.user_id	many-to-one	Each transaction is received by one user
transactions	category_id	transaction_categories.category_id	many-to-one	Each transaction belongs to one category
transaction_merchant	transaction_id	transactions.transaction_id	many-to-one	Links merchant participation to a transaction
transaction_merchant	merchant_id	merchants.merchant_id	many-to-one	Links transaction to a merchant
system_logs	transaction_id	transactions.transaction_id	many-to-one(optional)	A transaction can generate multiple logs
transactions	transaction_id	system_logs.transaction_id	one-to-many	One transaction generates many system logs

users	user_id	transactions.sender_id	one-to-many	A user can send many transactions
users	user_id	transactions.receiver_id	one-to-many	A user can receive many transactions
merchants	merchant_id	transaction_merchant.merchant_id	one-to-many	A merchant participates in many transactions
transactions	transaction_id	transaction_merchant.transaction_id	one-to-many	A transaction can involve multiple merchants
transaction_category	category_id	transactions.category_id	one-to-many	A category classifies many transactions

The junction table which is the Transaction merchant signifies:

- A single user can make multiple transactions.
- A single transaction can come from different users.